Lab 1- Learning-SQL

IBD

To open the shell and activate SQL under XAMPP

Use the following command in CMD or Shell from XAMPP

1- **Run the Apache and MySQL servers via XAMPP**
2- **Open the shell o the CMD**
3- **Go to MYSQL directory on your computer**

C:\Users\user> cd C:\xampp\mysql\bin

4- **Use the commend to run MariaDB monitor**

C:\xampp\mysql\bin> mysql -u root -p -h 127.0.0.1

```
C:\xampp\mysql\bin>mysql -u root -p -h 127.0.0.1
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

SQL Commands of Relational Algebra

List available databases:

SHOW DATABASES;

Creating a database.

- *The general command for creating a database*

CREATE DATABASE database_name;

(can be all lower case but capitals for commands is convention to differentiate commands from values)

A specific example:

CREATE DATABASE soap_store;

Delete a database:

DROP DATABASE database_name;

e.g.:

```
DROP DATABASE soap_store;
```

Tell MySQL which database to work with.

```
USE database_name;
```

e.g.,

```
CREATE DATABASE soap_store;

USE soap_store;
```

Tables

A MySQL database is essentially a collection of Tables.

Wiki definition: "A Table is a collection of related data held in a structured format within a database."

You can think of a table as similar to an excel spreadsheet, where you have columns (vertically) and rows (horizontally):

Name	Breed	Age
Rover	Labrador	3
Dennis	Jack Russel	1
Daisy	Golden Retrieve	6

Whenever you create a new column, MySQL requires that you specify what kind of data can be used to populate it (i.e. number / string etc...), to avoid data inconsistencies such as:

Name	Breed	Age
Rover	Labrador	3
Dennis	Jack Russel	one
Daisy	Golden Retriever	6

There are [loads of different MySQL datatypes](https://dev.mysql.com/doc/refman/8.0/en/data-types.html). (See below for how to declare them)

Creating a table:

```
CREATE TABLE table_name
```

```
 (
  column_name data_type,
  column_name data_type
 );
```

e.g.:

```
CREATE TABLE cats
 (
  name VARCHAR(100),
  age INT(10)
 );
```

Return all tables in current database:

```
SHOW TABLES;
```

Return slightly more detailed info about columns in the specified table in the current database:

```
SHOW COLUMNS FROM table_name;
```

A shorter-hand alternative to `SHOW COLUMNS` is

```
DESC table_name;
```

Permanently delete Table (be careful!):

```
DROP TABLE table_name;
```

Adding data to Tables (`INSERT`).

```
INSERT INTO table_name
 ( column1Name, column2Name )
VALUES     ( value1, value2 );
```

e.g.:

```
INSERT INTO cats
         ( NAME,
           age )
VALUES     ( "Jetson",
```

```
        5 );
```

You can insert multiple rows of data in one command by chaining values in parentheses and separating them by commas:

```
INSERT INTO table_name
        (column_a, column_b)
VALUES     (value_a, value_b),
           (value_a, value_b),
           (value_a, value_b);
```

e.g.,:

```
INSERT INTO cats
        ( NAME,    age )
VALUES     ( "Rover", 3 ),
        ( "Dennis", 1 ),
        ( "Daisy", 6 ),
         (“sandy”, NULL);
```

Select from the table

```
SELECT * FROM cats;
+-----------+------+
| name      | age  |
+-----------+------+
| Blue      |    1 |
| peanut    |    2 |
| Butter    |    4 |
| Jelly     |    7 |
| Boris     |    0 |
| sandy     | NULL |
+-----------+------+
```

You can prevent columns from being null by specifying NOT NULL:

```
CREATE TABLE cats2
```

```
 (
   name VARCHAR(100) NOT NULL,
   age INT NOT NULL
 );
INSERT INTO cats2 (name) VALUES ('stinky');
SELECT * FROM cats2;
+--------+-----+
| name   | age |
+--------+-----+
| stinky |   0 |
+--------+-----+
```

Setting default values

You can set the default values for a table using the `DEFAULT` keyword:

```
CREATE TABLE cats3
 (
   name VARCHAR(20) DEFAULT 'no name provided',
   age INT DEFAULT 99
 );
```

Now whenever no value is provided for `name`, the table will default to ``'no name provided'`, and when no age is provided, it will default to `99`.

Be careful using DEFAULT without the NOT NULL constraint, because although it will default in the *absence* of a value being passed in, it *could* still take `null` as an explicitly passed value.

e.g.:

```
INSERT INTO cats3() VALUES();
INSERT INTO cats3 (name, age) VALUES (null, null);
```

...neither of the above insertions would return any errors, and they would both successfully insert values into the table:

```
 SELECT * FROM cats3;
+------------------+------+
| name             | age  |
```

```
+------------------+------+
| no name provided |   99 |
| NULL             | NULL |
+------------------+------+
```

There is a risk you might find yourself with null values in your code base, so its important to safeguard against them being unintentionally inserted. If you don't want null values, use NOT NULL!

By contrast, if that table were created also using the `NOT NULL` constraint, it would return an error whenever NULL were passed in:

```
CREATE TABLE cats4
 (
   name VARCHAR(20) NOT NULL DEFAULT 'no name provided',
   age INT NOT NULL DEFAULT 99
 );
INSERT INTO cats4() VALUES();
SELECT * FROM cats4;
+------------------+-----+
| name             | age |
+------------------+-----+
| no name provided |  99 |
+------------------+-----+
INSERT INTO cats4 (name, age) VALUES (null, null);
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near
'VALES (null, null)' at line 1
```

Primary Key

A primary key is a column in a table that is used as a unique identifier (meaning no two values in that column can be the same).

Setting a primary key:

```
 CREATE TABLE unique_cats
  (
    cat_id INT NOT NULL PRIMARY KEY,
    name VARCHAR(100),
    age INT
  );
```

-- or alternatively:

```
CREATE TABLE unique_cats
(
  cat_id INT NOT NULL,
  name VARCHAR(100),
  age INT,
  PRIMARY KEY (cat_id)
);
DESC unique_cats;
```

```
+--------+--------------+------+-----+---------+-------+
| Field  | Type         | Null | Key | Default | Extra |
+--------+--------------+------+-----+---------+-------+
| cat_id | int(11)      | NO   | PRI | NULL    |       |
| name   | varchar(100) | YES  |     | NULL    |       |
| age    | int(11)      | YES  |     | NULL    |       |
+--------+--------------+------+-----+---------+-------+
```

Notice the key column now tells you that the `cat_id` field is the primary key (`PRI`).

This table will not accept any two entries where the `cat_id` columns hold the same value, because they must be unique.

```
INSERT INTO unique_cats(cat_id, name, age) VALUES (1, 'sid', 3);

INSERT INTO unique_cats(cat_id, name, age) VALUES (1, 'dave', 2);

ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
```

So, because the cat_id fields are the same in both inserts, the second insert has returned an error, and not executed.

Auto-incrementing:

You can use `AUTO_INCREMENT` to automatically fill ascending `INT` values for primary keys:

```
CREATE TABLE unique_cats2
(
   cat_id INT NOT NULL AUTO_INCREMENT,
   name VARCHAR(100),
   age INT,
   PRIMARY KEY (cat_id)
 );
INSERT INTO unique_cats2(name, age) VALUES('skippy', 4);
INSERT INTO unique_cats2(name, age) VALUES('bravo', 3);
INSERT INTO unique_cats2(name, age) VALUES('barbara', 10);
SELECT * FROM unique_cats2;
```

```
+--------+---------+------+
| cat_id | name    | age  |
+--------+---------+------+
|      1 | skippy  |    4 |
|      2 | bravo   |    3 |
|      3 | barbara |   10 |
+--------+---------+------+
```

Above you can see how the table has automatically added ascending numbers in the cat_id field for each insertion.

Basic Syntax of ALTER TABLE

When using an AUTO_INCREMENT column, SQL automatically assigns a unique number starting from 1 (by default). However, you can modify the starting value using ALTER TABLE:

```
ALTER TABLE table_name action;
```

e.g.,

```
ALTER TABLE unique_cats2 AUTO_INCREMENT = 10;
```

Ensures that the first productID in the products table starts at 10 instead of 1.

READ DATA

SELECT is used to read information from a table.

The * character can be used to denote all columns from a table:

```
SELECT * FROM cats;
```

-- alternatively you can specify any number and combination of columns

-- and in any order:

```
SELECT name FROM cats;

SELECT age FROM cats;

SELECT cat_id FROM cats;

SELECT name, age FROM cats;

SELECT cat_id, name, age FROM cats;

SELECT age, breed, name, cat_id FROM cats;

SELECT cat_id, name, age, breed FROM cats;
```

Specify conditions for queries with the WHERE keyword:

The WHERE keyword can be used to specify particular conditions for the data you want to target. Its not unique to Reading, it can also be used for updating, deleting etc... too.

Select by age or name:

```
SELECT * FROM cats WHERE age=4;

SELECT * FROM cats WHERE name='Egg';
```

Strings by default are not case sensitive:

```
SELECT * FROM cats WHERE name='egG';
```

You can also specify conditions for queries based on relationships between columns:

Select only cats where they have the same age as their cat_id number:

```
SELECT cat_id, age FROM cats WHERE cat_id=age;
```

Relabel column names with the AS keyword:

```
SELECT column_name AS alias_name FROM cats;
```

e.g.: Select cat_id and name from cats, and relabel cat_id to id:

```
SELECT cat_id AS id, name FROM cats;
```

Select name and breed from cats and rename them respectively as cat name and kitty breed SELECT name AS 'cat name', breed AS 'kitty breed' FROM cats;

Task

1- SHOW DATABASES to list all the existing databases in your server.
2- Create database, called "**Southwind**"
3- Create a table "**products**" in the database "**southwind**".

```
productID    INT UNSIGNED    NOT NULL AUTO_INCREMENT,
productCode  CHAR(3)         NOT NULL DEFAULT '',
name         VARCHAR(30)     NOT NULL DEFAULT '',
quantity     INT UNSIGNED    NOT NULL DEFAULT 0,
price        DECIMAL(7,2)    NOT NULL DEFAULT 99999.99,
PRIMARY KEY  (productID)
```

4- Insert the following information the table Products

productID	productCode	name	quantity	price
1001	PEN	Pen Red	5000	1.23
1002	PEN	Pen Blue	8000	1.25
1003	PEN	Pen Black	2000	1.25
1004	PEC	Pencil 2B	10000	0.48
1005	PEC	Pencil 2H	8000	0.49
1006	PEC	Pencil HB	0	9999999.99

5- Retrieve all products that have a quantity greater than **5000**